

**Educação  
Profissional  
Paulista**

Técnico em  
**Ciência de  
Dados**

# **Bibliotecas: Pandas, NumPy, SciPy, Matplotlib e Seaborn**

## **NumPy - acesso, inspeção e índice**

Aula 4

Código da aula [DADOS]ANO1C2B3S20A4

**Bibliotecas: Pandas,  
NumPy, SciPy,  
Matplotlib e Seaborn**

## Mapa da Unidade 5 Componente 3

NumPy: manipulação  
de array.

semana  
**22**

Pandas: manipulação  
de datas.

semana  
**27**

semana  
**19**

SciPy.

semana  
**20**

**Você está aqui!**  
NumPy: acesso,  
inspeção e índice.

semana  
**29**

Matplotlib: estrutura.

**Bibliotecas: Pandas,  
NumPy, SciPy,  
Matplotlib e Seaborn**

## **Mapa da Unidade 5 Componente 3**

# **Você está aqui!**

NumPy - acesso, inspeção e índice

### **Aula 4**

Código da aula:  
[DADOS]ANO1C2B3S20A4

**20**



## Objetivos da aula

- Praticar o conceito de funções e métodos da biblioteca SciPy.



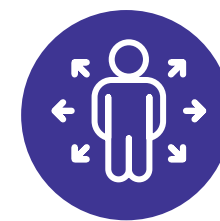
## Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



## Duração da aula

50 minutos.



## Competências técnicas

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões baseadas em evidências.



## Competências socioemocionais

- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

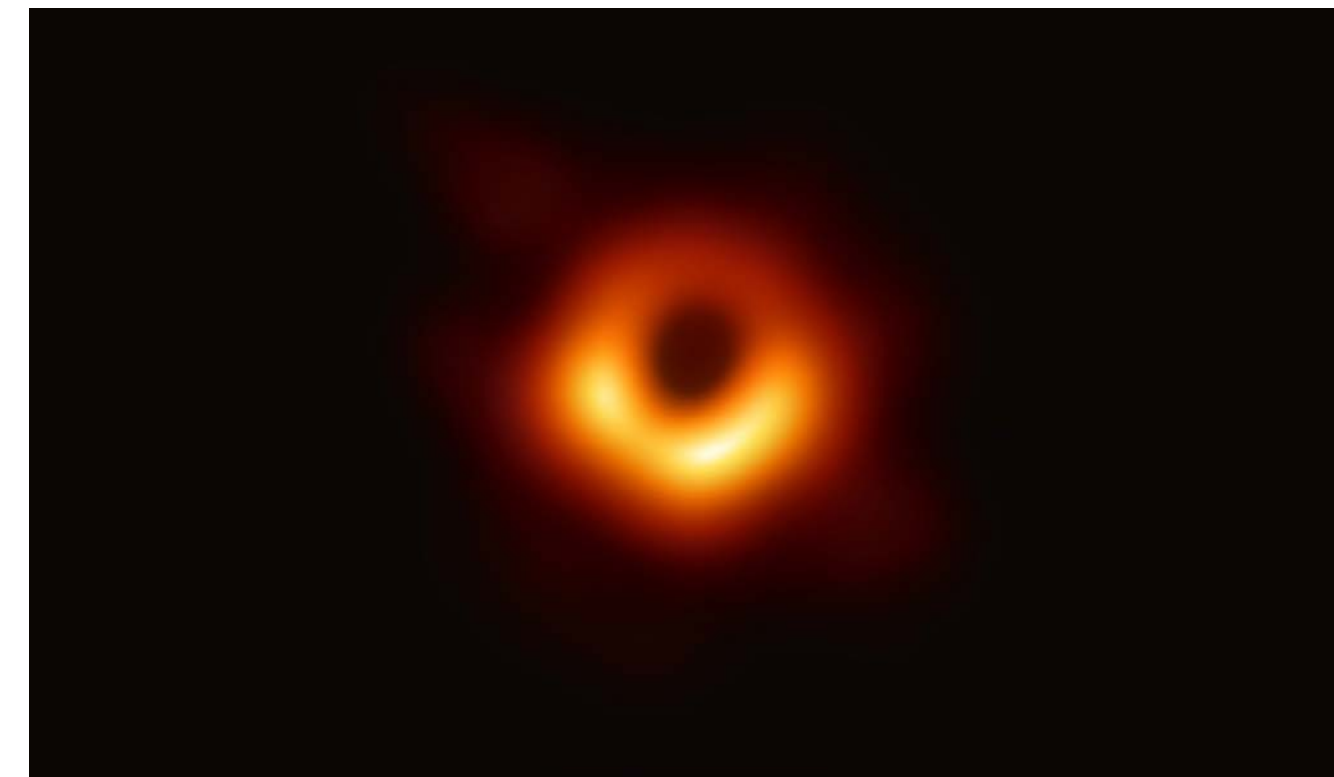


## Ponto de partida

# Estudo de Caso: a primeira imagem de um buraco negro

“A estrutura de dados n-dimensional que é a funcionalidade central do NumPy permitiu aos pesquisadores manipular grandes conjuntos de dados, fornecendo a base para a primeira imagem de um buraco negro.

Esse momento marcante na ciência fornece evidências visuais impressionantes para a teoria de Einstein.



Reprodução – ESO, 2022. Disponível em:  
<https://www.eso.org/public/images/eso2208-eh-t-mwd/>.  
Acesso em: 5 jun. 2024.

Esta conquista abrange não apenas avanços tecnológicos, mas colaboração científica em escala internacional entre mais de 200 cientistas e alguns dos melhores observatórios de rádio do mundo. Eles usaram algoritmos e técnicas de processamento de dados inovadores, que aperfeiçoaram os modelos astronômicos existentes, para ajudar a descobrir um dos mistérios do universo.”

(NUMPY, [s.d.]a)

# Construindo o conceito

## NumPy

Criamos um array no NumPy com `np.array()` e agora vamos inspecioná-lo.

```
1 import numpy as np
2
3 data = np.array([1, 2, 3])
4
5 # Número de dimensões
6 print("Número de dimensões:", data.ndim)
7
8 # Forma do array
9 print("Forma do array:", data.shape)
10
11 # Número total de elementos
12 print("Número total de elementos:", data.size)
13
14 # Tipo de dados dos elementos
15 print("Tipo de dados dos elementos:", data.dtype)
```

```
Número de dimensões: 1
Forma do array: (3,)
Número total de elementos: 3
Tipo de dados dos elementos: int32
```



### Tome nota

O array do NumPy deve ter todos os elementos do mesmo tipo – numéricos.

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo  
o **conceito**

# NumPy – Indexação e fatiamento

Você pode indexar e fatiar arrays do NumPy da mesma forma que pode fatiar listas do Python.

```
1 data = np.array([1, 2, 3])  
2  
3 print(data)  
4  
5 data
```

```
[1 2 3]  
array([1, 2, 3])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



## Construindo o conceito

# NumPy – Indexação e fatiamento

```
1 import numpy as np
2
3 # Criando o array
4 data = np.array([1, 2, 3])
5
6 # Exemplo 1: Acessando um elemento específico
7 print(data[0]) # Saída: 1
8 print(data[1]) # Saída: 2
9
10 # Exemplo 2: Fatiando para obter os dois primeiros elementos
11 print(data[0:2]) # Saída: [1, 2]
12
13 # Exemplo 3: Fatiando para obter todos os elementos a partir do segundo
14 print(data[1:]) # Saída: [2, 3]
15
16 # Exemplo 4: Fatiando para obter os dois últimos elementos
17 print(data[-2:]) # Saída: [2, 3]
```

1  
2  
[1 2]  
[2 3]  
[2 3]

| data |   | data[0] | data[1] | data[0:2] |
|------|---|---------|---------|-----------|
| 0    | 1 | 1       |         | 1         |
| 1    | 2 |         | 2       | 2         |
| 2    | 3 |         |         |           |

| data[1:] |   | data[-2:] |  |
|----------|---|-----------|--|
|          | 2 | 2         |  |
|          | 3 | 3         |  |

|   | data |    |  |
|---|------|----|--|
| 0 | 1    |    |  |
| 1 | 2    | -2 |  |
| 2 | 3    | -1 |  |
| 3 |      |    |  |

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Reprodução – NUMPY, [s.d.]c. Disponível em:  
[https://numpy.org/doc/stable/user/absolute\\_beginners.html](https://numpy.org/doc/stable/user/absolute_beginners.html). Acesso em: 5 jun. 2024.

## Construindo o conceito

# Exemplo: NumPy – Indexação e fatiamento

```
1 import numpy as np
2
3 # Criando diferentes arrays
4 data1 = np.array([1, 2, 3, 4, 5])
5 data2 = np.array([[1, 2, 3], [4, 5, 6]])
6 data3 = np.array([[1, 2], [3, 4], [5, 6]])
7
```

1. Qual é o último elemento do *array data1*?
2. Como você fatiaria o *array data1* para obter os três últimos elementos?
3. Qual é a primeira linha do *array data2*?
4. Como você fatiaria o *array data2* para obter os dois últimos elementos da segunda linha?
5. Qual é o elemento localizado na segunda linha e primeira coluna do *array data3*?
6. Como você fatiaria o *array data3* para obter a última coluna?
7. Como você acessaria simultaneamente a segunda linha do *array data1* e do *array data2*?
8. Como você fatiaria o *array data2* para obter a primeira e segunda linhas?
9. Como você inverteria a ordem dos elementos no *array data1*?
10. Como você selecionaria apenas os elementos maiores que 2 no *array data1*?

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

## Construindo o conceito

# Exemplo: NumPy – Indexação e fatiamento

```
8 # Exemplo 1: Acessando o último elemento de data1
9 print("Exemplo 1:")
10 print(data1[-1]) # Saída: 5
11
12 # Exemplo 2: Fatiando para obter os três últimos elementos de data1
13 print("Exemplo 2:")
14 print(data1[-3:]) # Saída: [3, 4, 5]
15
16 # Exemplo 3: Fatiando para obter a primeira linha de data2
17 print("Exemplo 3:")
18 print(data2[0]) # Saída: [1, 2, 3]
19
20 # Exemplo 4: Fatiando para obter os dois últimos elementos da segunda linha de data2
21 print("Exemplo 4:")
22 print(data2[1, -2:]) # Saída: [5, 6]
23
24 # Exemplo 5: Acessando o elemento na segunda linha e primeira coluna de data3
25 print("Exemplo 5:")
26 print(data3[1, 0]) # Saída: 3
27
28 # Exemplo 6: Fatiando para obter a última coluna de data3
29 print("Exemplo 6:")
30 print(data3[:, -1]) # Saída: [2, 4, 6]
```

Exemplo 1:  
5  
Exemplo 2:  
[3 4 5]  
Exemplo 3:  
[1 2 3]  
Exemplo 4:  
[5 6]  
Exemplo 5:  
3  
Exemplo 6:  
[2 4 6]

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

## Construindo o conceito

# Exemplo: NumPy – Indexação e fatiamento

```
32 # Exemplo 7: Acessando a segunda linha de data1 e data2 simultaneamente
33 print("Exemplo 7:")
34 print(data1[1], data2[1]) # Saída: 2 [4, 5, 6]
35
36 # Exemplo 8: Fatiando para obter a primeira e segunda linhas de data2
37 print("Exemplo 8:")
38 print(data2[:2]) # Saída: [[1, 2, 3], [4, 5, 6]]
39
40 # Exemplo 9: Fatiando para obter todos os elementos de data1 em ordem reversa
41 print("Exemplo 9:")
42 print(data1[::-1]) # Saída: [5, 4, 3, 2, 1]
43
44 # Exemplo 10: Fatiando para obter os elementos maiores que 2 de data1
45 print("Exemplo 10:")
46 print(data1[data1 > 2]) # Saída: [3, 4, 5]
```

Exemplo 7:  
2 [4 5 6]  
Exemplo 8:  
[[1 2 3]  
 [4 5 6]]  
Exemplo 9:  
[5 4 3 2 1]  
Exemplo 10:  
[3 4 5]

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



## Colocando em **prática**

### Exercício

$$\begin{bmatrix} 2 & 0 & 3 \\ 1 & 3 & -2 \\ 4 & 1 & 0 \end{bmatrix}$$

1. Crie um array NumPy a partir da matriz acima.
2. Inspecione o array recém-criado utilizando os atributos *ndim*, *shape*, *size* e *dtype*.
3. Acesse diferentes partes do array para responder as perguntas abaixo:
  - a) Qual é a primeira linha do array?
  - b) Qual é a última coluna do array?
  - c) Qual é a submatriz composta pelas duas primeiras linhas e duas primeiras colunas?
  - d) Quais são todos os elementos da segunda coluna?
  - e) Quais são os elementos da diagonal principal?
  - f) Qual é a última linha do array?



**Nesta aula**



**Em grupo**



Ser  
sempre +

## Título: “A jornada da foto pixelada”

Conheça Ana, uma jovem estudante de artes visuais. Ela adora explorar o mundo por meio de sua câmera, capturando momentos únicos e paisagens inspiradoras. Um dia, enquanto folheava um álbum antigo de família, Ana encontrou uma foto desgastada pelo tempo. Era uma imagem de seus avós em um piquenique na década de 1950.

A foto estava desbotada e com bordas rasgadas. As cores pareciam desvanecer, e os rostos dos avós estavam quase irreconhecíveis. Ana ficou intrigada. Ela sabia que essa foto guardava uma história especial, mas como poderia trazê-la de volta à vida?

Relato fictício elaborado especialmente para o curso.



Elaborado especialmente para o curso com apoio da ferramenta Microsoft Copilot.

Ser  
sempre +

## Título: “A jornada da foto pixelada”

Decidida a preservar essa memória, Ana digitalizou a foto em seu computador. Ao abrir o arquivo, ela se deparou com uma matriz de números. Era o *array numpy* que representava a foto pixelada. Ana se lembrou das aulas de programação e pensou: “Essa é minha chance de unir arte e tecnologia!”

Ela escreveu um código em Python para inspecionar o array. Percorreu cada elemento, analisando os valores de cor e brilho. À medida que avançava, os pixels ganhavam vida. Ela viu os tons de sépia da roupa dos avós, o verde do campo e até mesmo o sorriso tímido de sua avó.

Mas havia um problema. A foto estava corrompida. Alguns pixels estavam faltando, outros tinham valores errados. Ana precisava fatiar a imagem em pedaços menores e reconstruí-la. Ela usou técnicas de interpolação e preenchimento para preencher as lacunas.

À medida que trabalhava, Ana percebeu que a transformação da foto em um *array numpy* era como desvendar um enigma. Cada pixel era uma pista, e ela estava montando o quebra-cabeça da história de seus avós.



Ser  
sempre +

## Título: “A jornada da foto pixelada”

Ela se sentia como uma artista e uma cientista ao mesmo tempo.

Finalmente, após muitas horas de dedicação, Ana conseguiu restaurar a foto. As cores voltaram a brilhar, os contornos se tornaram nítidos. Ela havia desvendado a história por trás daquela imagem pixelada. E quando mostrou o resultado para seus pais e avós, todos se emocionaram ao reviver aquele momento especial.

Ana aprendeu que a transformação de uma foto em um *array numpy* não era apenas uma tarefa técnica. Era uma jornada de criatividade, paciência e imaginação. E assim, ela continuou sua busca por fotos antigas, ansiosa para desvendar mais histórias e preservar memórias para as gerações futuras.

- ▶ Será que essa história é possível? Fazem sentido as decisões tomadas por Ana?
- ▶ Você seguiria o mesmo caminho que Ana para solucionar os problemas encontrados?



© Getty Images

O que nós  
**aprendemos  
hoje?**

## Então ficamos assim...

- 1** Relembramos como inspecionar um array do NumPy.
- 2** Aprendemos sobre indexação e fatiamento do NumPy.
- 3** Vimos a história de Ana que pode tranquilamente ser real.

# Saiba mais

Está pronto para turbinar suas análises de dados com o poder do NumPy? Domine essa biblioteca essencial e realize cálculos numéricos com mais velocidade e precisão em Python.

ALURA. NumPy: análise numérica eficiente com Python. Disponível em:

<https://cursos.alura.com.br/course/numpy-analise-numerica-eficiente-pythons>. Acesso em: 4 jun. 2024.



# Referências da aula

MCKINNEY, W. *Python para análise de dados: tratamento de dados com Pandas, NumPy & Jupyter*. São Paulo: Novatec, 2023.

NUMPY. *Estudo de caso: a primeira imagem de um buraco negro*, [s.d.]a. Disponível em: <https://numpy.org/pt/case-studies/blackhole-image/>. Acesso em: 5 jun. 2024.

NUMPY. *Página inicial*, [s.d.]b. Disponível em: <https://numpy.org/pt/>. Acesso em: 4 jun. 2024.

NUMPY. *NumPy: the absolute basics for beginners*, [s.d.]c. Disponível em: [https://numpy.org/doc/stable/user/absolute\\_beginners.html](https://numpy.org/doc/stable/user/absolute_beginners.html). Acesso em: 4 jun. 2024.

Identidade visual: imagens © Getty Images.

**Educação  
Profissional  
Paulista**

Técnico em  
**Ciência de  
Dados**